ARENA: MULTI-AGENT REINFORCEMENT LEARNING

From Foundations to Emergent Intelligence: Architecture, Self-Play & Deep PPO Analysis

Author / Environment: Arena MARL Engine | **Framework:** Unity ML-Agents & Barracuda | **Status:** Complete Technical Dossier

---

Executive Brief: *This document provides a comprehensive, end-to-end reference for the* ***Arena*** *project. It covers the operational game loop, the visual physics engine, the fundamental principles of Reinforcement Learning (RL), Multi-Agent Reinforcement Learning (MARL), an exhaustive deep dive into* ***Proximal Policy Optimization (PPO)****, the 200-million-step evolutionary history of emergent tactics, and practical methodologies for benchmarking and visualizing AI progression against conventional scripted game bots.*

1. System Architecture & Game Loop

Arena is an interactive multi-agent reinforcement learning environment where two adversarial teams—**Hiders (Team 0, Cyan)** and **Seekers (Team 1, Crimson)**—compete on a bounded platform. Unlike static academic benchmarks, Arena introduces human environment modification and live co-play.

The Three-Phase Gameplay Loop

1. **The Architect Phase (Build Mode):** When a round begins in Test.unity, time is paused (Time.timeScale = 0). The human player acts as the arena architect with an energy budget of 100 points to place barricade walls (40 energy), spawn movable crates (25 energy), demolish existing pieces (35 energy), or pre-lock boxes to a team (20 energy). The resulting layout is persistently serialized to player_layout.json.
2. **The Grace Period (Warmup):** Once the round is unpaused, a tactical grace period initiates (the first 12% to 40% of the match). Seekers are frozen in place with laser scanners offline. Hiders take advantage of this head start to scramble across the room, push crates, block entryways, and find covert hiding positions.
3. **The Active Simulation Phase:** Seekers are unchained, deploying dynamic laser scanners and raycast vision cones. Hiders earn continuous positive rewards for remaining occluded from all seeker sightlines. Seekers earn rewards for maintaining line-of-sight and tagging hiders. The round concludes when the time limit expires or all hiders are captured.
4. **Player Co-Play (Possession):** Pressing **Tab** allows the player to possess an agent in real time, moving with WASD, grabbing/moving crates with E, and locking/unlocking crates with L/U. The remaining agents continue running their neural network inference policies.
5. **Multi-Angle Tactical Cameras:** The camera system provides 3 switchable modes: Isometric Tactical (Key 1), Top-Down Radar Overview (Key 2), and Dynamic Chase (Key 3), with full right-mouse orbit and mouse-wheel zoom.

2. Visual Physics & Scene Lighting Calibration

During initial playtesting, the arena appeared excessively dark and murky. An investigation of the Universal Render Pipeline (URP) and Physically Based Rendering (PBR) assets revealed four critical root causes:

- **Metallic Reflection Failure:** In PBR shaders, setting Metallic = 0.85 eliminates diffuse light, forcing the surface to reflect the surrounding environment. Because the camera background was set to solid near-black, surfaces reflected pure darkness. Lowering Metallic to 0.1 and Smoothness to 0.35 restored standard diffuse light absorption.
- **Albedo Base Color Crushing:** The floor material base color was configured to a dim navy (0.2, 0.25, 0.35), attenuating texture brightness by 75%. Resetting base colors to (0.55, 0.62, 0.72) made the cybernetic floor tiles crisp and visible.
- **Directional & Ambient Starvation:** Directional light was dimmed to 0.85 with a cold tint, while ambient light was minimal. Sun intensity was raised to 1.35 with a clean solar spectrum, and ambient probe intensity was boosted to 1.25.
- **Post-Processing Contrast & Vignette:** A post-processing volume applied high contrast (15) and a heavy vignette (0.38), crushing shadow details into pitch black. Contrast was normalized to 6, exposure lifted to +0.45, and vignette reduced to 0.18.

3. Reinforcement Learning: The Ground Truth

Reinforcement Learning (RL) is a computational approach to learning through goal-directed interaction. Unlike Supervised Learning (which requires labeled datasets of 'correct' moves) or Unsupervised Learning (which clusters unlabeled patterns), RL agents discover how to behave entirely by taking actions within an environment and observing scalar feedback: **rewards** and **penalties**.

Dimension	Traditional Scripted Game AI	Reinforcement Learning (RL) AI
Core Logic	Handcrafted Finite State Machines (FSM), Behavior Trees, or A* NavMesh heuristics.	Deep Neural Networks mapping high-dimensional sensor data to action probability distributions.
Adaptation	Rigid and predictable. If a player places a wall, NavMesh pathfinding breaks unless re-baked.	Dynamic generalization. The agent reacts to unexpected geometry using continuous 360° ray sensors.
Development	Developer manually programs every behavior (e.g., 'if seeker near, crouch behind crate').	Developer designs only the reward function; agents discover complex tactics through trial and error.
Discovery	Zero emergent discovery. The AI cannot do anything the programmer did not anticipate.	Emergent strategies. Discovers unintended tool use, room sealing, and coordinated team plays.

The Mathematical Formulation: Markov Decision Process (MDP)

An RL environment is formalized as a Markov Decision Process defined by the tuple **(S, A, P, R, γ)**:
• **State Space (S):** The state at time t, s_t, contains all relevant platform observations (agent positions, velocities, rotation angles, crate coordinates, team IDs).
• **Action Space (A):** Continuous movement vectors (X, Z thrust and yaw rotation) plus discrete branch actions (Grab/Release Box, Lock/Unlock Box).
• **Transition Dynamics (P):** Unity PhysX engine computing the physical state of the next step s_{t+1} given action a_t.
• **Reward Function (R):** Scalar reward r_t given to agents based on visibility, survival, capture, and out-of-bounds penalties.
• **Discount Factor ($\gamma = 0.99$):** Weights immediate vs. future rewards, prioritizing long-term survival over short-term gains.

4. Multi-Agent RL (MARL) & Self-Play Dynamics

When multiple autonomous learners operate in the same shared environment, the problem transitions from an MDP to a **Markov Game (Stochastic Game)**. This introduces two profound phenomena: **Non-Stationarity** and **Co-Evolution**.

The Challenge of Non-Stationarity

In single-agent RL, the environment is stationary: a given action in state s always yields predictable reward distributions. In MARL, because the opponents are simultaneously learning and modifying their neural policies, an action that worked yesterday (e.g., hiding behind a pillar) fails today because the Seekers learned to flank that pillar. The ground is constantly moving beneath each agent's feet.

Self-Play & The Competitive Arms Race

To achieve robust mastery without human demonstrations, the environment utilizes **Self-Play**. Instead of training against a fixed bot, agents train against current and historical versions of themselves. In the project's training configuration (arena_train.yaml), the self-play engine maintains an Elo rating matrix (initialized at 1200.0) and periodically swaps opponents with past checkpoints (swap_steps: 2000).

The Self-Play Arms Race Loop:
1. Generation N: *Hiders discover they can run away from Seekers → Hider win-rate climbs.*
2. Generation N+1: *Seekers adapt, learning coordinated flanking to corner Hiders → Seeker win-rate climbs.*
3. Generation N+2: *Hiders discover movable crates and block doorways → Hiders regain dominance.*
4. Generation N+3: *Seekers learn to exploit gaps in barricades and rush during the grace period → Equilibrium shifts.*

5. Deep Dive: Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO), published by Schulman et al. (OpenAI, 2017), is the state-of-the-art policy gradient algorithm powering Arena. It combines the sample efficiency of first-order optimization with the stability of trust-region methods.

The Policy Gradient Dilemma & Policy Collapse

In basic Policy Gradient methods (such as REINFORCE), the agent updates its neural network weights θ along the gradient of expected reward: $\nabla J(\theta) = E[\nabla_\theta \log \pi_\theta(a|s)\, Q(s,a)]$. However, standard gradient descent has a fatal flaw in reinforcement learning: **the step size dilemma**.

• If the step size (learning rate) is too small, training takes years.

• If the step size is slightly too large, the neural network weights can shift radically. Unlike supervised learning where bad data only causes a momentary spike in loss, a bad policy update in RL causes the agent to collect catastrophic, degraded experience in the next round. The agent forgets everything it learned, falling off a performance cliff into **policy collapse**.

Why TRPO Needed a Successor

OpenAI previously solved this with **Trust Region Policy Optimization (TRPO)**, which enforced a strict statistical boundary using Kullback-Leibler (KL) divergence: $KL(\pi_{old}, \pi_{new}) \leq \delta$. While mathematically robust, TRPO required computing second-order derivatives (the Fisher Information Matrix) and running conjugate gradient optimization. It was computationally heavy, slow, and incompatible with complex architectures. **PPO was engineered to achieve the exact same trust-region stability using simple, elegant first-order gradient clipping.**

THE PPO CLIPPED SURROGATE OBJECTIVE FUNCTION

$$L^{CLIP}(\theta) = \hat{E}_t \left[\min\left(r_t(\theta)\, \hat{A}_t ,\ \ \text{clip}\left(r_t(\theta) , 1 - \varepsilon , 1 + \varepsilon \right) \hat{A}_t \right) \right]$$

Deconstructing the Formula Term by Term

1. The Probability Ratio $r_t(\theta)$:

$r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta old}(a_t|s_t)$

This measures how much more or less likely the new policy makes action a_t compared to the old policy. If $r_t(\theta) = 1$, the policies are identical. If $r_t(\theta) = 1.3$, the new policy is 30% more likely to take that action.

2. The Advantage Function $\hat{A}_t$:

$\hat{A}_t = Q(s_t, a_t) - V(s_t)$

The advantage tells us whether a specific action was *better or worse than average*. The Critic network estimates the baseline value $V(s)$ (how good the current position is). If the actual cumulative return exceeds $V(s)$, $\hat{A}_t$ is positive (a great move!). If it underperforms, $\hat{A}_t$ is negative.

3. The Clipping Mechanism clip(r_t, 1 - ε, 1 + ε):

The parameter ε (set to 0.2 in arena_train.yaml) defines our safety boundary: [0.8, 1.2]. It acts as a strict governor on how much the policy can change in a single update.

4. The Min Operator (The Pessimistic Bound):

Why take the minimum between the unclipped and clipped objective? This creates a conservative lower bound:

• **Case A ($\hat{A}_t > 0$, Great Action):** We want to increase the probability of this action. The ratio r_t grows. However, once r_t exceeds $1 + \varepsilon$ *(1.2)*, the clipping caps the term at $(1 + \varepsilon)\hat{A}_t$. The gradient becomes zero! The network is prohibited from becoming recklessly overconfident on a single lucky roll.

• **Case B ($\hat{A}_t < 0$, Catastrophic Action):** We want to decrease the probability. The ratio r_t drops. If r_t drops below $1 - \varepsilon$ *(0.8)*, clipping stops pushing the gradient once the penalty is accounted for, preventing the distribution from collapsing to zero.

The Complete Actor-Critic Loss Function

In production (including Unity ML-Agents), PPO does not just optimize the policy. It trains an **Actor-Critic** architecture sharing representation layers, combining three distinct objectives into a single unified loss function:

$$L^{TOTAL}(\theta) = \hat{E}_t [L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

- **Value Function Loss $L^{VF}(\theta)$:** Squared error between the Critic's state estimation and actual discounted returns: $(V_\theta(s_t) - V_{target})^2$. Trains the Critic to accurately evaluate board states.
- **Entropy Bonus $S[\pi_\theta]$:** Measures the randomness/entropy of the policy distribution. In arena_train.yaml, beta: 0.005 rewards exploration, ensuring agents don't prematurely converge on a sub-optimal repetitive loop before discovering tool use.
- **Generalized Advantage Estimation (GAE, $\lambda = 0.95$):** Blends temporal difference residuals over time horizon (*time_horizon: 64*), balancing bias and variance in advantage calculations.

Config Hyperparameters Decoded (from arena_train.yaml)

Hyperparameter	Value in Config	Engineering Purpose
batch_size	1024	Number of experience transitions sampled per gradient descent update.
buffer_size	10240	Experiences collected across parallel environments before running policy updates.
learning_rate	0.0003 (linear decay)	Adam optimizer learning rate, decreasing toward zero as models mature.
beta (entropy)	0.005	Strength of entropy bonus to encourage agents to try novel actions.
epsilon (clip)	0.2	PPO clipping threshold [0.8, 1.2], restricting radical policy jumps.
lambd (GAE)	0.95	GAE smoothing parameter trading off immediate reward variance vs. long-term bias.
num_epoch	3	Number of full passes over the buffer data during each training cycle.
max_steps	2,000,000 (train) / 200M (catalog)	Total training budget before final neural weights are exported to ONNX.

6. The Four Evolutionary Eras of Emergence

By analyzing the model checkpoints in Unity Project/Assets/NNModels, we can trace the historical emergence of intelligence across 200 Million simulation steps:

Era / Steps	Primary Behaviors	Strategic Significance
Era 1: Toddler Phase (0 – 50K Steps)	Random twitching, continuous collisions with walls, aimless wandering. Objects are ignored.	Untrained neural baseline. Random weights; gradient vectors establishing basic motor correlation.
Era 2: Chaser Phase (1M – 10M Steps)	Seekers run straight toward Hiders; Hiders sprint in reverse. Resembles simple playground tag.	Policy discovers basic objective rewards (proximity and capture), but lacks spatial depth.
Era 3: Occlusion Phase (20M – 75M Steps)	Hiders realize walls break seeker vision cones. Seekers learn corner-peeking and angle clearance.	Emergence of spatial geometry awareness. Agents utilize static walls to manipulate sightlines.
Era 4: Fortification Phase (100M – 200M Steps)	**Emergent Tool Use:** During the grace period, Hiders drag crates into doorways and trigger lockTeam. Seekers learn to probe barricade seams.	**Peak MARL Intelligence:** Construction and shelter building invented purely through self-play survival pressure. No human coded this logic.

7. Demonstrating AI Superiority Over Traditional Game AI

To convince an observer that Arena is powered by genuine Reinforcement Learning and not scripted bot logic, the system provides three concrete verification pillars:

1. **Generational A/B Testing (GenerationComparer.cs):** Pressing **[G]** opens the checkpoint comparer. On the identical room layout, pitting an Era 1 checkpoint (5k steps) against Seekers results in an instant 100% capture rate within 5 seconds. Switching to the 200M FCSP checkpoint immediately shifts the hidden survival rate to >85%, demonstrating mathematical policy convergence.
2. **The Dynamic Architect Stress Test:** Scripted bots rely on pre-baked NavMesh waypoints. If an obstacle is introduced at runtime, scripted bots freeze or clip into walls. In Arena, the player can build asymmetric mazes during the Architect Phase; because agents perceive continuous physics via 360° raycasts, they navigate and barricade custom rooms with zero re-baking.
3. **Real-Time Telemetry & Perception Rays:** Traditional bots hide internal states. In Arena, seeker laser scanners (SeekerLaserScanner.cs), tactical rings (AgentTacticalRing.cs), and ray perception sensors can be visualized in real time, exposing how the neural net evaluates vision cones and decision outputs.